

STUDENT PERFORMANCE WORKSHEET PSEUDOCODE

1. START PROGRAM

START

SET student_records TO the predefined list of student records

SET valid_students, invalid_records TO

 CALL validate_records(student_records)

CALL ASSIGN_GRADES(valid_students)

WHILE TRUE

 DISPLAY "STUDENT PERFORMANCE WORKSHEET SOFTWARE"

 DISPLAY "FCC GROUP 2"

 DISPLAY "1. View Valid Students"

 DISPLAY "2. View Invalid Records"

 DISPLAY "3. Analyse Class Performance"

 DISPLAY "4. View Intervention List"

 DISPLAY "5. Search For a Student"

 DISPLAY "6. Exit"

INPUT selection

IF selection = 1 THEN

 CALL DISPLAY_VALID_STUDENTS(valid_students)

ELSE IF selection = 2 THEN

 CALL DISPLAY_INVALID_RECORDS(invalid_records)

ELSE IF selection = 3 THEN

 CALL DISPLAY_CLASS_PERFORMANCE(valid_students)

ELSE IF selection = 4 THEN

CALL VIEW_INTERVENTION_LIST(valid_students)

ELSE IF selection = 5 THEN

CALL SEARCH_STUDENT(student_records)

ELSE IF selection = 6 THEN

DISPLAY "Program closed."

BREAK

ELSE

DISPLAY "Invalid selection."

END IF

END WHILE

END

2. VALIDATE STUDENT RECORDS

FUNCTION VALIDATE_RECORDS(records)

SET valid_students TO empty list

SET invalid_records TO empty list

FOR EACH record IN records

SET is_valid TO TRUE

SET reasons TO empty list

Check Student ID

IF student ID is missing THEN

SET is_valid TO FALSE

ADD "Missing Student ID" TO reasons

END IF

Check Programme

IF programme is missing THEN

SET is_valid TO FALSE

ADD "Missing Programme" TO reasons

END IF

Check Project Submission Status

IF submitted_project is NOT Boolean THEN

SET is_valid TO FALSE

ADD "Project status is not Boolean" TO reasons

END IF

Check marks and attendance

FOR EACH field IN

assignment_mark,

cat_mark,

exam_mark,
attendance_pct

IF field is NOT numeric THEN

SET is_valid TO FALSE

ADD "Field is not numeric" TO reasons

ELSE IF field < 0 OR field > 100 THEN

SET is_valid TO FALSE

ADD "Field is outside the range 0 to 100" TO reasons

END IF

END FOR

IF is_valid = TRUE THEN

ADD record TO valid_students

ELSE

ADD record AND reasons TO invalid_records

END IF

END FOR

RETURN valid_students, invalid_records

END FUNCTION

3. CALCULATE FINAL MARKS

FUNCTION CALCULATE_FINAL_MARKS(valid_students)

FOR EACH student IN valid_students

SET final_mark TO

(assignment_mark $\times$ 0.20)

+ (cat_mark $\times$ 0.20)

+ (exam_mark $\times$ 0.60)

ROUND final_mark TO 2 decimal places

SET student final_mark TO final_mark

END FOR

RETURN valid_students

END FUNCTION

4. ASSIGN GRADES

FUNCTION ASSIGN_GRADES(valid_students)

FOR EACH student IN valid_students

IF student final_mark $\geq$ 70 THEN

SET grade TO "A"

ELSE IF student final_mark $\geq$ 60 THEN

SET grade TO "B"

ELSE IF student final_mark >= 50 THEN

SET grade TO "C"

ELSE IF student final_mark >= 40 THEN

SET grade TO "D"

ELSE

SET grade TO "E"

END IF

SET student grade TO grade

END FOR

RETURN valid_students

END FUNCTION

5. DISPLAY VALID STUDENTS

FUNCTION DISPLAY_VALID_STUDENTS(valid_students)

DISPLAY "--- VALID STUDENT RECORDS ---"

IF valid_students is empty THEN

DISPLAY "No valid records found."

ELSE

FOR EACH student IN valid_students

 DISPLAY student details

END FOR

END IF

DISPLAY "-----"

END FUNCTION

6. DISPLAY INVALID RECORDS

FUNCTION DISPLAY_INVALID_RECORDS(invalid_records)

 DISPLAY "--- INVALID STUDENT RECORDS ---"

 IF invalid_records is empty THEN

 DISPLAY "No invalid records found."

 ELSE

 FOR EACH record IN invalid_records

 DISPLAY "Student ID: ", student ID

 DISPLAY "Reason(s): ", invalid reason(s)

 END FOR

 END IF

 DISPLAY "-----"

END FUNCTION

7. ANALYSE CLASS PERFORMANCE

FUNCTION CALCULATE_CLASS_STATISTICS(valid_students)

SET total_marks TO 0

SET highest_final_mark TO first student's final mark

SET lowest_final_mark TO first student's final mark

SET grade_A TO 0

SET grade_B TO 0

SET grade_C TO 0

SET grade_D TO 0

SET grade_E TO 0

FOR EACH student IN valid_students

SET total_marks TO

total_marks + student final_mark

IF student final_mark > highest_final_mark THEN

SET highest_final_mark TO student final_mark

END IF

IF student final_mark < lowest_final_mark THEN

SET lowest_final_mark TO student final_mark

END IF

IF student grade = "A" THEN

 INCREMENT grade_A BY 1

ELSE IF student grade = "B" THEN

 INCREMENT grade_B BY 1

ELSE IF student grade = "C" THEN

 INCREMENT grade_C BY 1

ELSE IF student grade = "D" THEN

 INCREMENT grade_D BY 1

ELSE

 INCREMENT grade_E BY 1

END IF

END FOR

SET total_students TO number of students in valid_students

IF total_students > 0 THEN

 SET class_average TO

 total_marks / total_students

ELSE

SET class_average TO 0

END IF

RETURN class_average,

highest_final_mark,

lowest_final_mark,

total_students,

grade_A,

grade_B,

grade_C,

grade_D,

grade_E

END FUNCTION

8. DISPLAY CLASS PERFORMANCE

FUNCTION DISPLAY_CLASS_PERFORMANCE(valid_students)

SET class statistics TO

CALL CALCULATE_CLASS_STATISTICS(valid_students)

DISPLAY "--- CLASS PERFORMANCE SUMMARY ---"

DISPLAY "Class Average: ", class_average

DISPLAY "Highest Final Mark: ", highest_final_mark

DISPLAY "Lowest Final Mark: ", lowest_final_mark

DISPLAY "Total Students: ", total_students

DISPLAY "Grade A: ", grade_A

DISPLAY "Grade B: ", grade_B

DISPLAY "Grade C: ", grade_C

DISPLAY "Grade D: ", grade_D

DISPLAY "Grade E: ", grade_E

DISPLAY "-----"

END FUNCTION

9. VIEW INTERVENTION LIST

FUNCTION VIEW_INTERVENTION_LIST(valid_students)

SET intervention_found TO FALSE

DISPLAY "--- INTERVENTION LIST ---"

FOR EACH student IN valid_students

SET reasons TO empty list

IF student final_mark < 50 THEN

ADD "Final mark below 50" TO reasons

END IF

IF student attendance_pct < 75 THEN

ADD "Attendance below 75%" TO reasons

END IF

IF student submitted_project = FALSE THEN

ADD "Project not submitted" TO reasons

END IF

IF reasons is NOT empty THEN

SET intervention_found TO TRUE

DISPLAY "Student ID: ", student ID

DISPLAY "Programme: ", programme

DISPLAY "Final Mark: ", final mark

DISPLAY "Attendance: ", attendance percentage

DISPLAY "Intervention Reasons:"

FOR EACH reason IN reasons

DISPLAY reason

END FOR

END IF

END FOR

IF intervention_found = FALSE THEN

DISPLAY "No students require intervention."

END IF

END FUNCTION

10. SEARCH FOR A STUDENT

FUNCTION SEARCH_STUDENT(records)

INPUT student_id

WHILE student_id is not numeric AND student_id is not "#"

 DISPLAY "Invalid ID. Numbers only are required."

 INPUT student_id

END WHILE

IF student_id = "#" THEN

 RETURN

END IF

SET student_id TO "ST" + student_id formatted to 3 digits

SET student_found TO FALSE

FOR EACH student IN records

 IF student ID = student_id THEN

 SET student_found TO TRUE

DISPLAY "--- STUDENT DETAILS ---"

DISPLAY student ID

DISPLAY programme

DISPLAY assignment mark

DISPLAY CAT mark

DISPLAY examination mark

DISPLAY attendance percentage

DISPLAY project submission status

IF final mark exists THEN

 DISPLAY final mark

END IF

IF grade exists THEN

 DISPLAY grade

END IF

BREAK

END IF

END FOR

IF student_found = FALSE THEN

 DISPLAY "Student not found."

END IF

END FUNCTION

11. END PROGRAM

END